

Техническое задание

Обработчик бизнес-документации

1. Описание задачи

Разработать функционал обработки документов из заданной директории, который:

- **Рекурсивно** обходит все вложенные папки.
- Извлекает **только файлы** с расширениями `.doc` и `.pdf`.
- Обрабатывает файлы:
 - `.doc` файлы — с использованием специального обработчика.
 - `.pdf` файлы — если можно прочитать, используем доступный метод. Если нет, задействуем **Mistral OCR**.
- Преобразует содержимое в **JSON** (по уже имеющейся логике `document_processor`).
- Преобразует файлы в **source_files** и **paragraphs** для хранения в базе данных.
- Создаёт **векторные представления** параграфов с помощью Supabase Edge Function API:

<https://xhppqnkewoqhcg1ccgkq.supabase.co/functions/v1/victorize>.

- Загружает обработанные данные в базу согласно структуре, описанной в `20240311_create_tables.sql`.

2. Функциональные требования

1. Чтение и обработка файлов

- На вход передаётся **абсолютный путь** к папке с документацией.
- Из директории **рекурсивно** извлекаются все `.doc` и `.pdf` файлы.
- Файлы `.doc` обрабатываются специализированным обработчиком.
- Файлы `.pdf`:
 - Если можно **прочитать**, используем стандартный метод.
 - Если нет — **Mistral OCR**.

2. Преобразование содержимого

- Каждый файл преобразуется в **JSON**, используя логику `document_processor`.
- Данные разбиваются на **source_files** и **paragraphs**.

3. Векторизация (Embedding)

- Каждый параграф отправляется в Supabase Edge Function API **victorize**.
- Возвращённые эмбединги сохраняются в базе.

4. Загрузка в базу данных

- Данные загружаются в соответствии со структурой **20240311_create_tables.sql**.

3. Технические детали

3.1. Структура JSON

Формат JSON-объекта должен соответствовать тому, что уже используется в **document_processor**.

3.2. База данных

- Данные разбиваются на **source_files** (информация о файле) и **paragraphs** (разбитое содержимое).
- Векторные эмбединги загружаются в соответствующую таблицу.

3.3. Используемые технологии

- **Python** для основной обработки.
- **python-docx** для **.doc** файлов.
- **pdfplumber** для **.pdf** файлов.
- **Mistral OCR** для сложных PDF.
- **Requests** для взаимодействия с Supabase Edge Function API.
- **PostgreSQL + Supabase** для хранения данных.

4. Входные и выходные данные

4.1. Входные данные

- Абсолютный путь к папке с документами (**пример: /Users/eslinko/Downloads/FoliPharm/**).

4.2. Выходные данные

1. **JSON-файлы**, соответствующие структуре **document_processor**.
 2. **Записи в базе данных** (source_files, paragraphs).
 3. **Векторные представления**, загруженные в Supabase.
-

5. Ошибки и обработка исключений

- **Неверный путь к директории** → Ошибка и логирование.
 - **Файл не может быть обработан** (например, битый PDF) → Логирование ошибки, но обработка продолжается.
 - **Ошибка при загрузке векторных эмбеддингов** → Повторная попытка, если неудачно — логирование.
-

6. Этапы разработки

1. Реализовать **рекурсивный обход папки** и сбор **.doc** и **.pdf** файлов.
 2. Реализовать **обработку .doc** файлов.
 3. Реализовать **обработку .pdf** файлов:
 - Читаемые PDF — через стандартный обработчик.
 - Нечитаемые PDF — через **Mistral OCR**.
 4. Разбить файлы на **параграфы** и **source_files**.
 5. Преобразовать данные в **JSON**.
 6. Векторизовать параграфы через Supabase Edge Function.
 7. **Сохранить данные в базу**.
 8. Обеспечить **обработку ошибок и логирование**.
-

7. Дополнительные требования

- Код должен быть **читаемым и документированным**.
- **Логирование** действий в процессе обработки.
- Возможность **запуска из командной строки (CLI)**.